potentially become a backing service for another application. That’s what usually happens in a cloud native system.

2.1.12 Stateless processes

In the previous chapter, you saw that high scalability is one reason why we move to the cloud. To ensure scalability, we design applications as stateless processes and adopt a *share-nothing architecture* : no state should be shared among different application instances. Ask yourself if any data would be lost should an instance of your application be destroyed and recreated. If the answer is affirmative, then your application is not stateless.

No matter what, we will always need to save some state, or our applications will be useless in most cases. As a result, we design applications to be stateless and then only handle the state in specific stateful services like data stores. In other words, a stateless application delegates the state management and storage to a backing service.

2.1.13 Concurrency

Creating stateless applications is not enough to ensure scalability. If you need to scale, that means you need to serve more users. Therefore, your applications should allow concurrent processing to serve many users at the same time.

The 15-Factor methodology defines processes as first-class citizens. Those processes should be horizontally scalable, distributing the workload across many processes on different machines, and this concurrent processing is only possible if the applications are stateless. In JVM applications, we handle concurrency through multiple threads, available from thread pools.